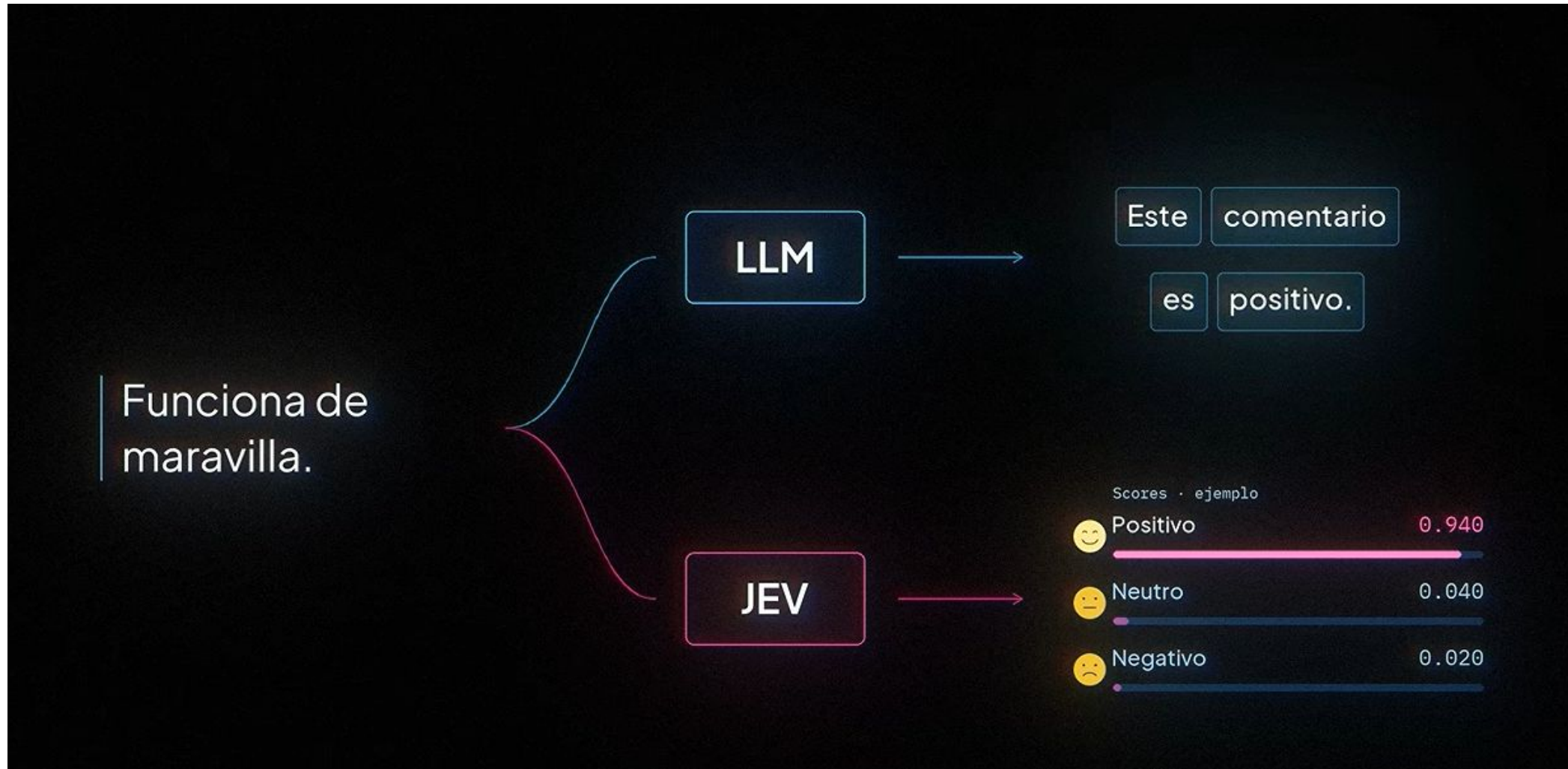


Ingeniería de soluciones con AI

Semestre 2026-2

Docente:
Francisco Macaya

Noticias: Jev, el nuevo modelo generalista.



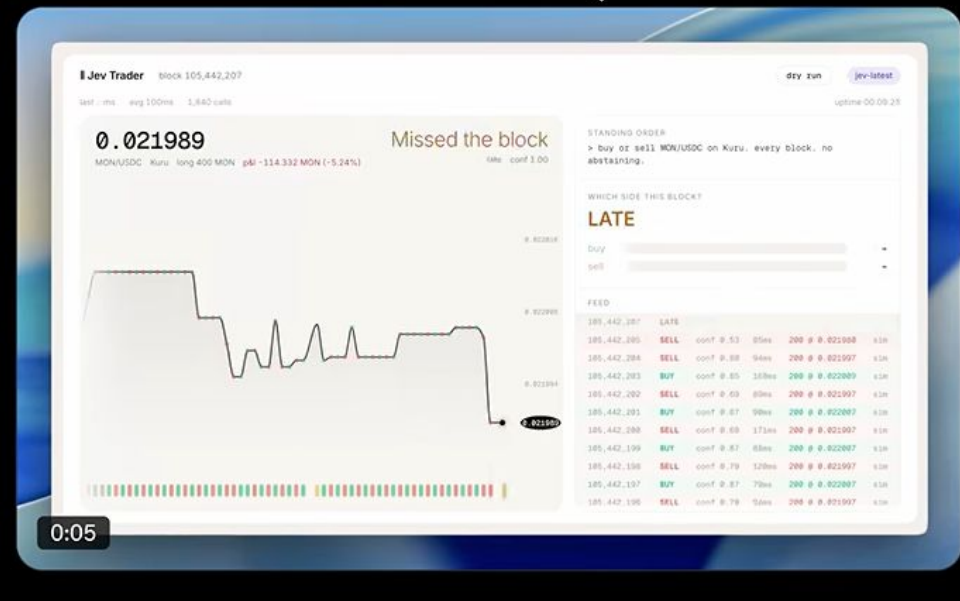
Noticias: Jev, el nuevo modelo generalista.

Mostrar traducción

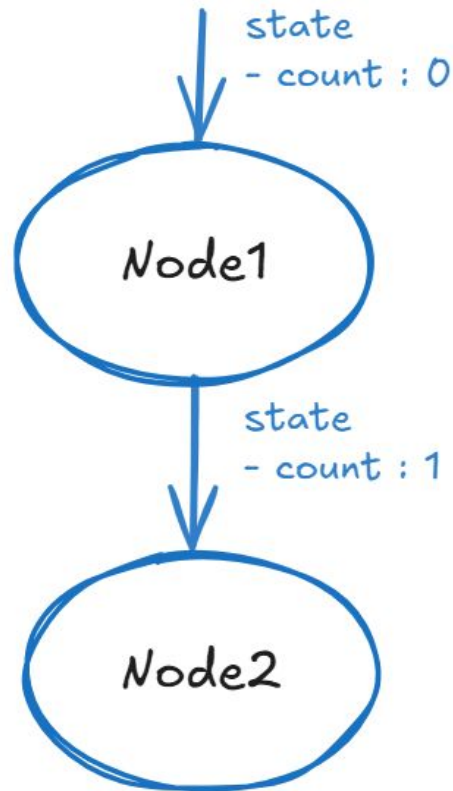
Jev is INSANE.

I built this in an evening and morning. A real-time trading bot ingesting onchain+offchain data to make rapid decisions about trades. Fully autonomous.

So far it has lost me \$31,680.

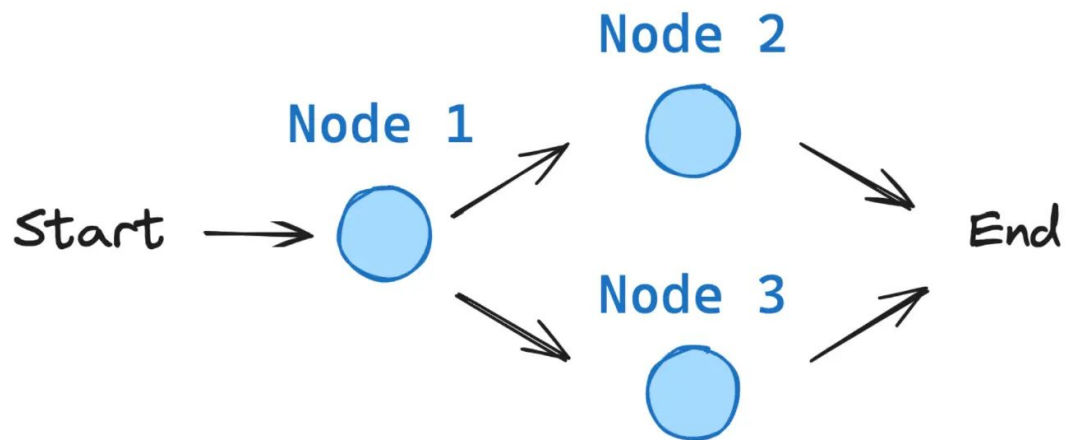


LangGraph: States, información entre conexiones.



El estado es un diccionario tipado que viaja por todo el grafo. Cada nodo recibe el estado completo y devuelve solo los campos que quiere cambiar.

LangGraph: Nodo, un paso del proceso



Nodo es una función python, recibe el estado, hace algo y devuelve una actualización parcial.

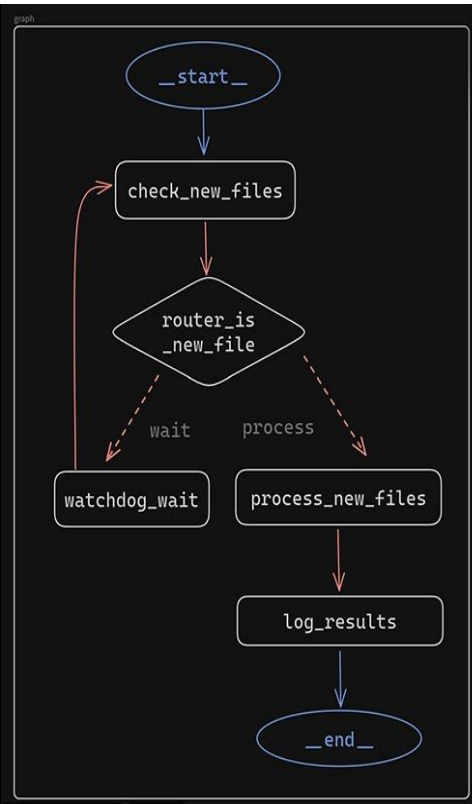
El nodo puede llamar a un LLM, una base de datos, un cálculo o devolver nada si no cambia el estado.

LangGraph: Edge, conexiones entre nodos.

LangGraph Conditional Edges

```
workflow.add_conditional_edges(
    "check_new_files",
    router_is_new_file,
    {
        "process": "process_new_files",
        "wait": "watchdog_wait",
    },
)
```

skool.com/business24ai



Arista fija

Siempre va al mismo lugar. "Después de buscar, vuelve al agente":

```
graph.add_edge("rag_tools", "rag_agent")
```

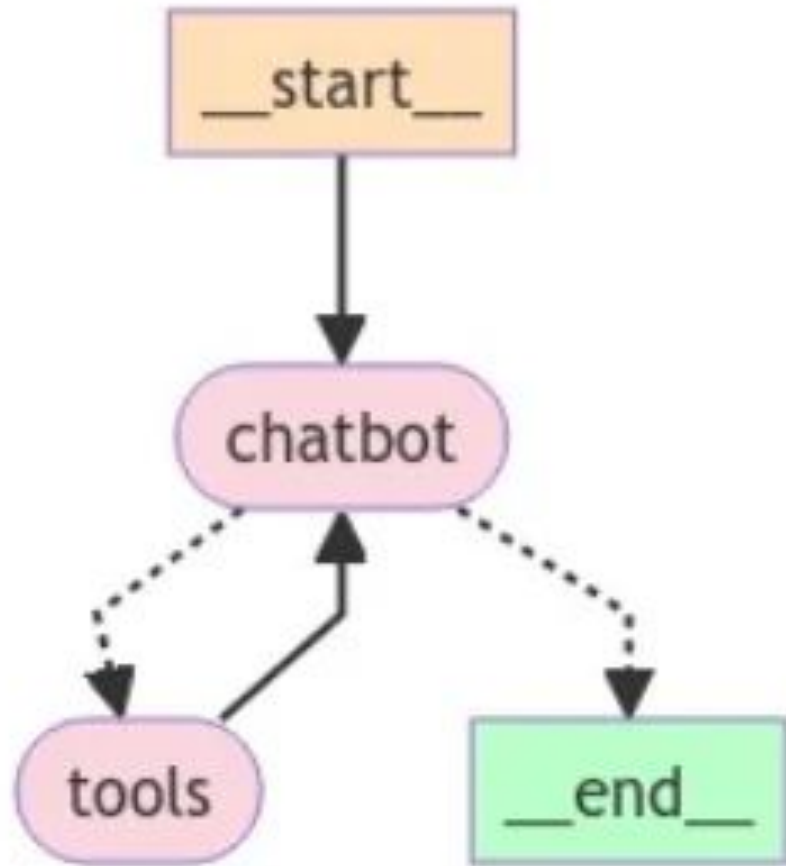
Arista condicional

Una función mira el estado y **devuelve el nombre** del siguiente nodo:

```
def route_supervisor(state: AgentState) -> str:
    return {"rag": "rag_agent", "meeting": "meeting_agent"}.get(state["destino"], END)

graph.add_conditional_edges("supervisor", route_supervisor, {
    "rag_agent": "rag_agent",
    "meeting_agent": "meeting_agent",
    END: END,
})
```

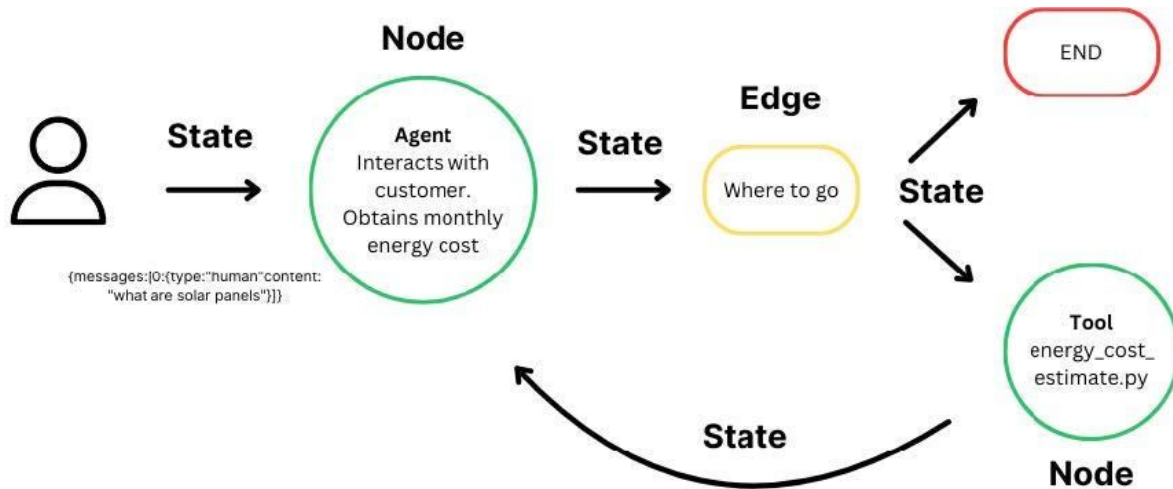
LangGraph: Tools, capacidades al LLM.



Una tool es una función decorado con @tool. El LLM no ejecuta nada, solo pide que se ejecute, y LangGraph la ejecuta.

El docString no es la documentación, es el prompt de la tool. LangChain construye un esquema JSON a partir del nombre y el docString.

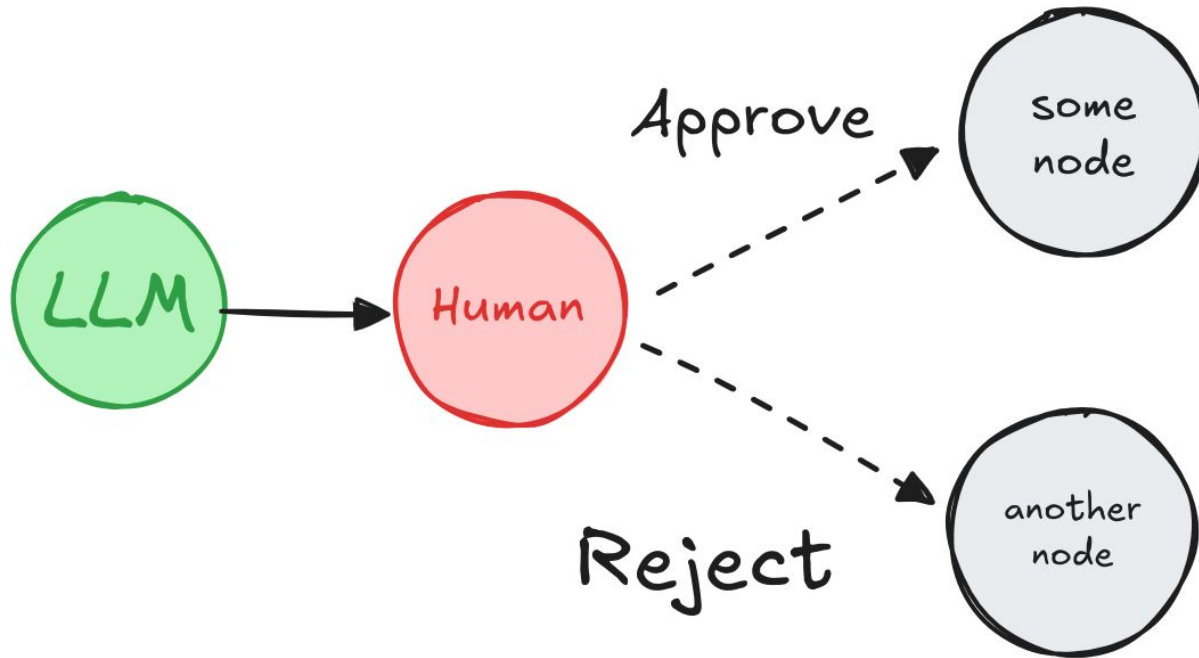
LangGraph: Compile, del plano a ejecutable.



El compile() valida el plano, y devuelve un objeto ejecutable.

```
app = graph.compile(checkpointer=checkpointer)
```

LangGraph: Checkpoint y interrupt



Sin checkpoint, cada invoke empieza desde 0, el agente no recuerda nada del turno anterior.

El checkpoint guarda una foto del estado después de cada paso, agrupando por `thread_id`.

Interrupt, es un nodo que suspende la ejecución y espera a una persona (solo funciona si existe un checkpoint).

El reanudar no vuelve a pasar por el nodo inicial, sino que donde se interrumpio.

LangGraph: ¿Cómo encaja todo?

```
graph = StateGraph(AgentState)           # 1. el plano, con su estado

graph.add_node("supervisor", supervisor) # 2. los pasos
graph.add_node("rag_agent", rag_agent)
graph.add_node("rag_tools", ToolNode(rag_tools))

graph.set_entry_point("supervisor")      # 3. por dónde empieza

graph.add_conditional_edges(             # 4. las decisiones
    "supervisor", route_supervisor,
    {"rag_agent": "rag_agent", END: END},
)
graph.add_edge("rag_tools", "rag_agent") # 5. los caminos fijos

app = graph.compile(checkpointer=MemorySaver()) # 6. a ejecutable, con memoria
```